

---

# Ejercicios - OOP

## Ejercicio 1

Explicar las características de las siguientes metodologías para el diseño de "software":

- Behavior-Driven Design (BDD)
- Test-Driven Design (TDD)
- Component-Based Design (CBD)
- Service Architecture Oriented (SAO)

### a) Behavior-Driven Design (BDD)

Este diseño consiste en nombrar las pruebas de software en función del comportamiento esperado del sistema usando un lenguaje común (DSL, domain-specific language) que pueda ser entendido por todos los participantes en un proyecto (desarrolladores, testers, etc.).

Se basa en la interacción entre los distintos roles del equipo de desarrollo para definir el comportamiento del sistema a través de ejemplos concretos para formalizar un conocimiento compartido.

Es útil en situaciones en las que se requiere una estrecha colaboración entre los equipos técnicos y de negocio para asegurar que los requisitos se entienden y se implementan correctamente.

### b) Test-Driven Development (TDD)

Esta metodología se basa en escribir pruebas automatizadas con el objetivo de que fallen para luego mejorar el código ("refactoring") y repetir el proceso.

Es eficiente para generar un código limpio y de calidad basado en módulos pequeños y bien definidos. Inicialmente requiere más tiempo pero a la larga compensa el esfuerzo.

### c) Component-Based Design (CBD)

Este diseño se centra en la creación de sistemas a partir de componentes modulares e independientes, parecido al TDD

Cada componente tiene una interfaz bien definida y encapsula su propio estado y comportamiento, lo que facilita su reemplazo en caso de errores.

Es útil en sistemas grandes que requieren modularidad y donde la reutilización de componentes es importante.

## d) Service-Oriented Architecture (SOA)

Es un modelo de arquitectura de software que organiza las funcionalidades del sistema como servicios independientes y autónomos.

Es indicado para sistemas integrados y cuando los servicios son proporcionados a través de una red a otros sistemas o aplicaciones.

Un servicio tiene cuatro propiedades principales:

- representa una funcionalidad específica del negocio repetible,
- es autónomo y auto-contenido, por lo que puede ser desplegado y ejecutado de manera independiente,
- es una "caja negra" para sus consumidores ya que no necesitan conocer su funcionamiento interno,
- puede estar compuesto por otros servicios.

## Ejercicio 2

### a) What is an abstract class?

Una clase abstracta es aquella que no puede ser instanciada directamente y está diseñada para ser heredada por otras clases. De esta manera podemos decir que actúa de plano ("blueprint" en inglés).

Siguiendo el ejemplo visto en clase, la clase "Casa" sería abstracta ya que luego se crean clases hijas como "Casa con piscina" o "Casa con jardín". El ejemplo del primer enlace de la diapositiva 62 habla sobre como hacer un programa para el pago de empleados a tiempo completo y a tiempo parcial, y la clase abstracta es la de "Empleado".

### b) How can you implement an abstract class in Python?

En python se puede implementar una clase abstracta usando el módulo abc (Abstract Base Classes) que permite definir métodos abstractos. En este apartado voy a hacer explícito el ejemplo del pago de empleados del mismo enlace de la diapositiva 62:

```
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, first_name, last_name):
        self.first_name = first_name
        self.last_name = last_name

    @property
    def full_name(self):
        return f"{self.first_name} {self.last_name}"

    @abstractmethod
    def get_salary(self):
        pass
```

Aquí se define la clase abstracta "Employee" que tiene un método abstracto "get\_salary" el cual debe ser implementado por las clases hijas.

```
class FulltimeEmployee(Employee):
    def __init__(self, first_name, last_name, salary):
        super().__init__(first_name, last_name)
        self.salary = salary

    def get_salary(self):
        return self.salary

class HourlyEmployee(Employee):
    def __init__(self, first_name, last_name, worked_hours, rate):
        super().__init__(first_name, last_name)
        self.worked_hours = worked_hours
        self.rate = rate

    def get_salary(self):
        return self.worked_hours * self.rate
```

Posteriormente se calculan los salarios de los empleados con la clase "Payroll".

```
class Payroll:
    def __init__(self):
        self.employee_list = []

    def add(self, employee):
        self.employee_list.append(employee)

    def print(self):
        for e in self.employee_list:
            print(f"{e.full_name} \t ${e.get_salary()}")
```

Finalmente se imprimen los resultados mediante la creación de instancias de las clases hijas.

```
payroll = Payroll()

payroll.add(FulltimeEmployee('John', 'Doe', 6000))
payroll.add(FulltimeEmployee('Jane', 'Doe', 6500))
payroll.add(HourlyEmployee('Jenifer', 'Smith', 200, 50))
payroll.add(HourlyEmployee('David', 'Wilson', 150, 100))
payroll.add(HourlyEmployee('Kevin', 'Miller', 100, 150))

payroll.print()
```

Con el siguiente resultado:

|               |         |
|---------------|---------|
| John Doe      | \$6000  |
| Jane Doe      | \$6500  |
| Jenifer Smith | \$10000 |
| David Wilson  | \$15000 |
| Kevin Miller  | \$15000 |

El enlace mencionado de la diapositiva 62 es el siguiente:

<https://www.pythontutorial.net/python-oop/python-abstract-class/>

## Ejercicio 3

### a) EXERCISE-A

#### a.i) Define the class diagram for the following classes:

Diagrama de clases en formato textual (UML, Unified Modeling Language) para las clases indicadas en la diapositiva 30 en la Figura 1.

#### a.ii) Create the class diagram for the following classes and identify multiple inheritance if any: prey lion animal mammal fish deer predator

Diagrama de clases en formato textual (UML, Unified Modeling Language) para las clases indicadas en la diapositiva 31 en la Figura 2.

#### a.iii) Design a library system using OOP

Diagrama de clases en formato textual (UML, Unified Modeling Language) para un sistema de biblioteca usando OOP en la Figura 3.

Webs consultadas:

- <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/>
- <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-class-diagram/>

### b) EXERCISE-B

#### b.i) What is the MRO of the classes?

MRO (Method Resolution Order por sus siglas en inglés) es el orden en el que Python busca métodos y atributos en una jerarquía de clases con herencia múltiple.

#### b.ii) Fix the bug

En el siguiente código hay un error:

```
class ShoppingCart:
    def __init__(self, items=[]):
        self.items = items

    def add_item(self, item):
        self.items.append(item)

    def display_items(self):
        print("Items in the cart:", self.items)

cart1 = ShoppingCart()
cart2 = ShoppingCart()

cart1.add_item("Apple")
cart1.add_item("Banana")

cart2.add_item("Orange")
cart2.add_item("Grapes")

cart1.display_items()
cart2.display_items()
```

El código resulta en la siguiente salida:

```
Items in the cart: ['Apple', 'Banana', 'Orange', 'Grapes']
Items in the cart: ['Apple', 'Banana', 'Orange', 'Grapes']
```

```
# Vamos a arreglar el problema causado por el uso de una lista mutable  
    como valor por defecto en el constructor de la clase ShoppingCart.  
# Para hacerlo vamos a cambiar el valor por defecto de items a None y  
    luego inicializaremos una nueva lista dentro del constructor si  
    items es None.
```

```
class ShoppingCartFixed:
    def __init__(self, items=None):
        if items is None:
            items = []
        self.items = items

    def add_item(self, item):
        self.items.append(item)

    def display_items(self):
        print("Items in the cart:", self.items)

cart1_fixed = ShoppingCartFixed()
cart2_fixed = ShoppingCartFixed()

cart1_fixed.add_item("Apple")
cart1_fixed.add_item("Banana")

cart2_fixed.add_item("Orange")
cart2_fixed.add_item("Grapes")

cart1_fixed.display_items()
cart2_fixed.display_items()
```

La salida corregida es la que sigue:

```
Items in the cart: ['Apple', 'Banana']
Items in the cart: ['Orange', 'Grapes']
```

### b.iii) Static overload in python

```
def add(a, b):
    return a + b
def add(a, b, c):
    return a + b + c

print(add(2, 4))
```

La salida del código anterior es la siguiente:

```
TypeError: add() missing 1 required positional argument: 'c'
```

Lo que ocurre es que la segunda definición de la función "add" sobrescribe la primera, por lo que cuando se llama a "add(2, 4)", Python espera tres argumentos en lugar de dos.

#### b.iv) What does it mean \*args and \*\*kwargs?

Los argumentos \*args permiten a una función aceptar un número variable de argumentos posicionales, mientras que los \*\*kwargs de palabras clave. Esto es útil cuando se desconoce el número de argumentos que se pasarán a la función.

```
def f(x):
    return x**2/2.0 + 3

def g(x):
    return 10/3*x - x**3

# Using a function as agreement, defining another functions inside,
# and return the defined function -> DECORATOR

def a_warning( funct ):
    def wrapper(*args, **kwargs):
        print('Just warning you...')
        return funct(*args, **kwargs)
    return wrapper

# Assign to a variable the decorated function
f_w = a_warning( f )
print( f_w(23) )

# Equivalently, using the @ syntax
@a_warning
def f_w( x ):
    return x**2/2.0 + 3

print( f_w(23) )
```

La salida del código es la siguiente:

```
Just warning you...
267.5
Just warning you...
267.5
```

que se repite debido a se ha definido dos veces la función decorada.

Las funciones decorativas son una forma de modificar o ampliar el comportamiento de una función sin cambiar su código fuente. Se definen como funciones que toman otra función como argumento, y devuelven una nueva función que envuelve la función original, añadiendo una funcionalidad adicional.

La web consultada es la siguiente:

[https://python-intermedio.readthedocs.io/es/latest/args\\_and\\_kwargs.html](https://python-intermedio.readthedocs.io/es/latest/args_and_kwargs.html)

## b.v) functools - functions as higher order objects

```
def factorial(n):
    return n * factorial(n - 1) if n else 1

def g(x):
    return 10/3*x - x**3

# Calculate factorial(13) -> 14 recursive calls
# Then calculate factorial(7) -> Just get the cached result
# Then calculate factorial(15) -> cached result plus 2 more recursive calls
```

He investigado el módulo functools de Python, que sirve para funciones de orden superior, es decir, funciones que operan sobre otras funciones.

La web consultada es la siguiente:

<https://docs.python.org/es/3/library/functools.html>

## b.vi) Lambda functions

Usamos una función lambda para ordenar una lista de ntuplas

```
my_list = [(1,2), (23,5), (2,8), (4,6), (9,32)]
sorted_list = sorted(my_list, key=lambda x: x[1]) # Sorting by the
second element of each tuple, x[0] for the first element
print(sorted_list)
```

y el resultado imprimido es el siguiente:

```
[(1, 2), (23, 5), (4, 6), (2, 8), (9, 32)]
```

## b.vii) Iterators - Generators

Los iteradores son objetos que permiten recorrer una secuencia de datos sin necesidad de almacenar todos los elementos en memoria. Los generadores son una forma especial de iteradores que permiten crear secuencias de datos bajo demanda utilizando la palabra clave "yield".

Mediante el uso de clases se puede crear un generador de números primos, como se pide en la diapositiva 51:

```
# Ahora voy a hacer lo mismo pero con clases:
class PrimeGenerator:
    def __init__(self, limit):
        self.limit = limit
        self.num = 2

    def is_prime(self, num):
        if num < 2:
            return False
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0:
                return False
        return True
```

```

def __iter__(self):
    return self

def __next__(self):
    while self.num < self.limit:
        if self.is_prime(self.num):
            prime = self.num
            self.num += 1
            return prime
        self.num += 1
    raise StopIteration

# Usage
prime_gen = PrimeGenerator(50)
for prime in prime_gen:
    primes = []
    for p in prime_generator(50):
        primes.append(p)

print(primes)

```

El resultado imprimido de este código es una tupla de números primos menores que 50, que he decido imprimir así por simplicidad:

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

## Apéndice

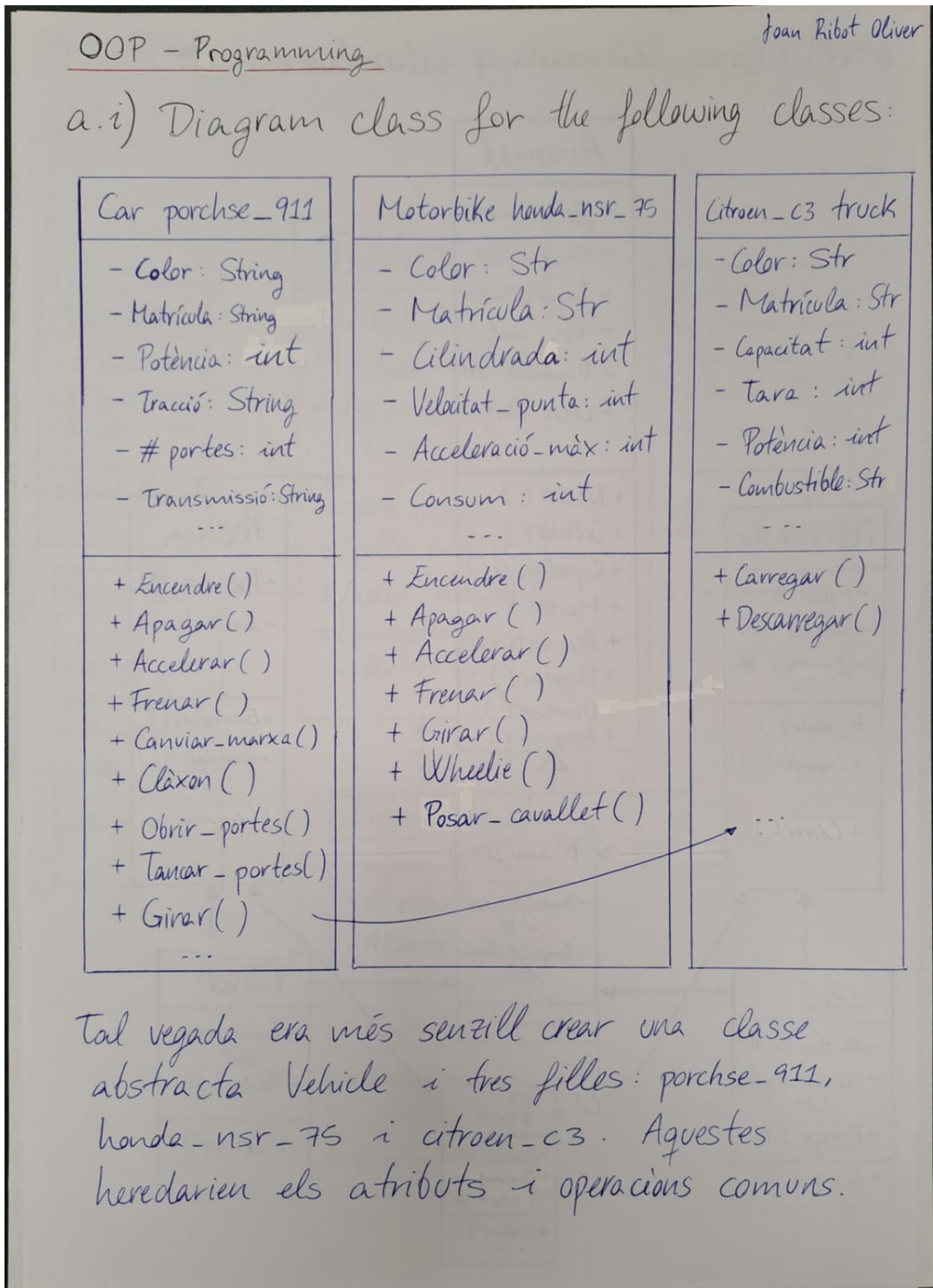


Figure 1: a.i) class diagram for the following classes: car porchse\_911, motorbike honda\_nsr\_75, citroen\_c3 truck.

## a.ii) Class diagram + inheritance

Joan Thibot Oliver

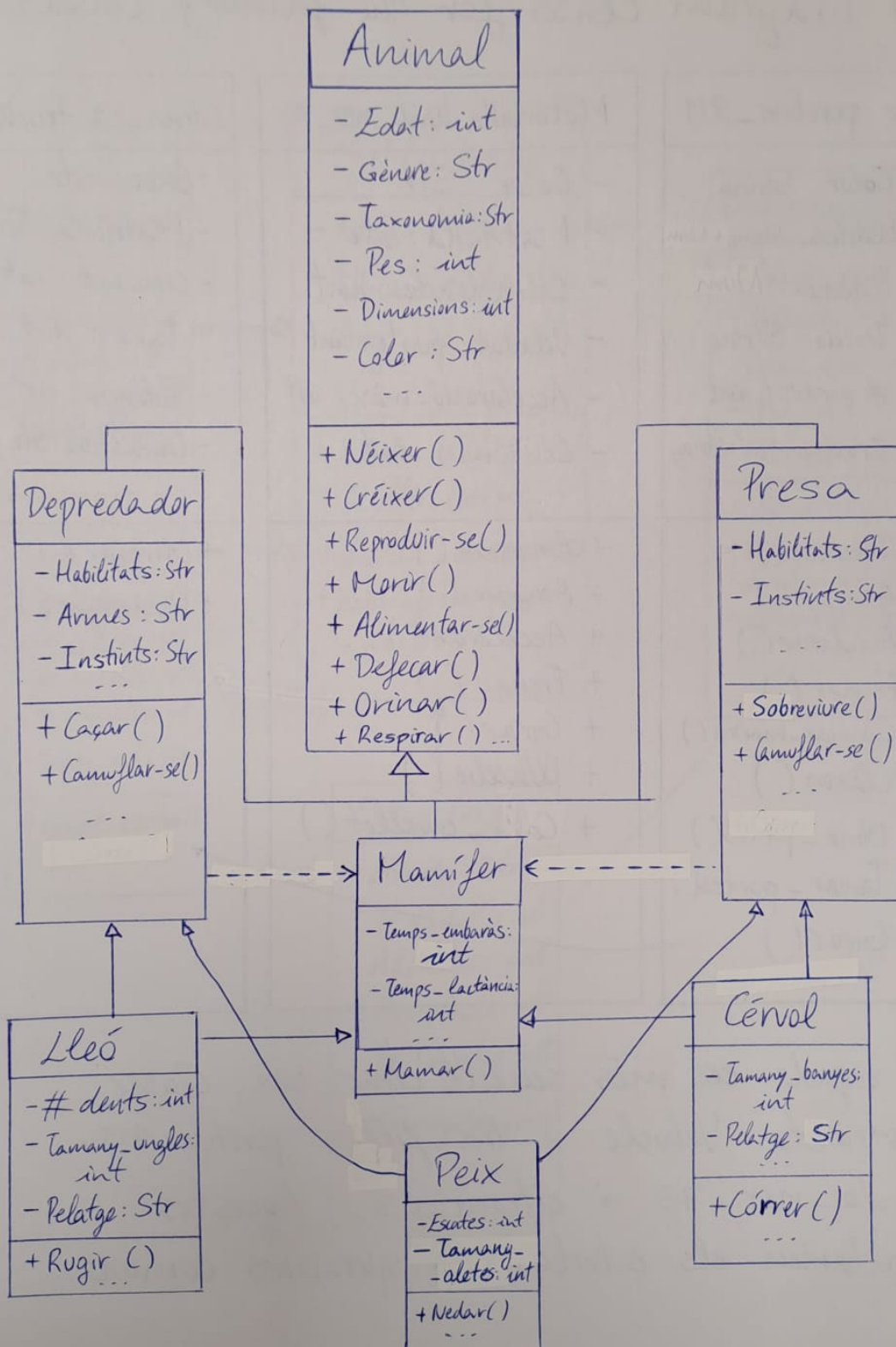


Figure 2: a.ii) class diagram for the following classes and their heritances: prey lion animal mammal fish deer predator.

(a.iii) Design a library system  
UML using OOP.

Joan Ribot Oliver

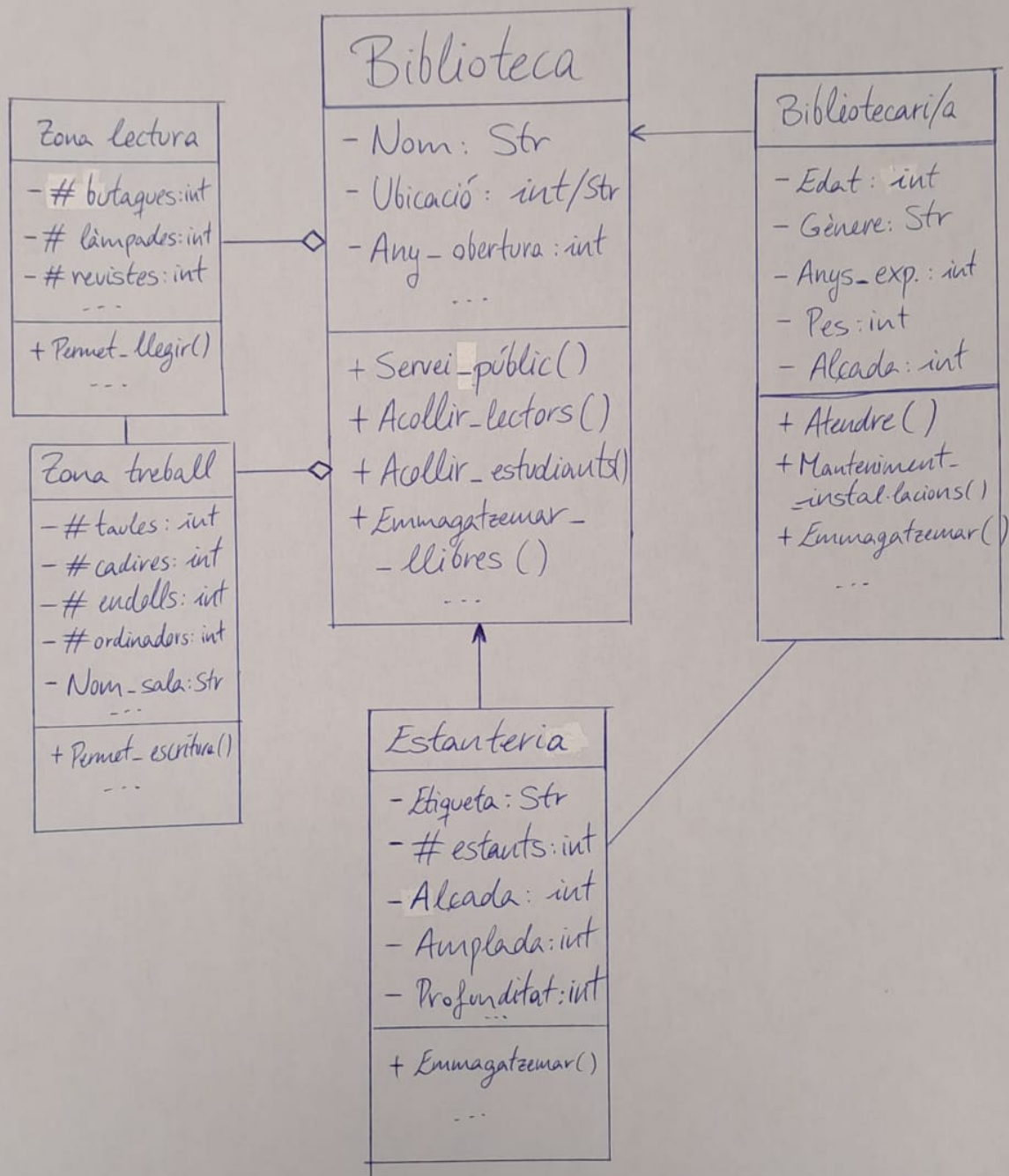


Figure 3: a.iii) class diagram for a library system using OOP.